A Novel State Space Exploration Method for the Sparse-Reward Reinforcement Learning Environment

Xi Liu[1], Long Ma[1], Zhen Chen[1], Changgang Zheng[2], Ren Chen[3], Yong Liao[1], and Shufan Yang[4](✉)

[1] University of Science and Technology of China, Hefei, China
yliao@ustc.edu.cn
[2] University of Oxford, Oxford, UK
[3] Anhui Medical University, Hefei, China
[4] Edinburgh Napier University, Edinburgh, UK
S.Yang@napier.ac.uk

Abstract. Sparse-reward reinforcement learning environments pose a particular challenge because the agent receives infrequent rewards, making it difficult to learn an optimal policy. In this paper, we propose NSSE, a novel approach that combines that stratified state space exploration with prioritised sweeping to enhance the informativeness of learning, thus enabling fast learning convergence. We evaluate NSSE on three typical Atari sparse reward environments. The results demonstrate that our state space exploration method exhibits strong performance compared to two baseline algorithms: Deep Q-Network (DQN) and noisy Deep Q-Network (Noisy DQN).

Keywords: Sparse-reward · Replay Sub-buffers · DQN · Exploration · Reinforcement Learning

1 Introduction

Reinforcement learning (RL) conducts agent learning by interacting with the environment and trial errors. Specifically, by sensing rewards from the environment, the RL policy will be updated regularly over time in order to maximise cumulative rewards. Not all environment provides informative rewards, which is known as the sparse reward problem. Under the sparse reward environment, agents mainly receive rewards with the same value, barely gaining feedback with valid values [1]. The lack of informative feedback leads to inefficient agent exploration and diffused policy optimisation, ultimately hindering RL convergence. Addressing the challenge of sparse rewards is an active research area in the reinforcement learning paradigm. The most widely used methods, mainly based on experience replay buffers [8,13,17], can mitigate the effect but are resources

Dr Yang is supported by Royal Academy Engineering SHE project RAEng (IF2223-172) and Royal Society of Edinburgh (961_Yang).

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023
M. Bramer and F. Stahl (Eds.): SGAI 2023, LNAI 14381, pp. 216–221, 2023.
https://doi.org/10.1007/978-3-031-47994-6_18

consuming, especially in terms of storage and computational. In this paper, we propose Novel State Space Exploration (NSSE), a new experience replay method with prioritised sweeping on multi-buffer for the sparse reward environment. This method reduces computational and storage requirements compared to approaches with prioritised storage and improves the sample efficiency of reinforcement learning algorithms.

2 Background

Sparse Reward Problem. The sparse reward challenge in reinforcement learning arises from the sparsely distributed rewards in the environment, posing a training difficulty for deep RL [1]. A classic approach to address this issue involves the integration of intrinsic rewards, which augment exploration by introducing dense rewards. Typically, these intrinsic rewards are generated based on training experiences, encompassing factors like novelty [4,9], curiosity [10,15], or information theory [14,16]. While these methods enhance exploration, they may introduce training challenges such as derailment and detachment [5].

Prioritised Sweeping. Prioritised sweeping is a common technique used for addressing Markov Decision Problems, which assigns update priorities based on temporal difference errors (TD-error) magnitudes. Saglam et al. [12] applied this approach to actor-critic algorithms. Prioritised sweeping in replay memory selects experiences by prioritising those with higher anticipated learning potential on future iterations. Schaul et al. [13] implemented this prioritisation mechanism within the replay buffer to increase the replay frequency of more critical experiences. Horgan et al. [8] expanded prioritised experience replay to distributed architectures for large-scale training, yielding effective learning with high-dimensional data. Nonetheless, prioritised experience replay shares limitations with its reliance on historical experiences. While it proves especially beneficial in scenarios featuring sparse reward signals, it necessitates frequent priority calculations and updates, slowing down the training process and consuming more memory.

3 Problem Formulation

We consider a Markov Decision Process (MDP) [11] formulated as ($\mathcal{S}$, $\mathcal{A}$, $\mathcal{P}$, $\mathcal{R}$, γ), where $\mathcal{S}$ is the set of state, $\mathcal{A}$ is the set of actions, $\mathcal{P}(s'|s,a)$ is the state-transition probability function, $\mathcal{R}(s,a)$ is the immediate reward from state s to state s' given an action, and $\gamma \in [0,1]$ is the discount factor. At time t, the agent observes the state s_t, takes an action a_t; and then obtains an immediate reward $r(s_t, a_t)$ and a new state ${s_t}'$, determined from the probability distribution $\mathcal{P}$. A stochastic policy π indicates the action taken by the agent in the case of given s. The objective of the agent is to find the policy π mapping from states to actions that maximise the discounted cumulative sum of random rewards formulated as $Z^{\pi}(s,a) = \sum_{i=t}^{\infty} \gamma^{i-t} r(s_i, a_i)$. The expected value is $Q^{\pi}(s,a) = E\left[Z^{\pi}(s,a)\right]$ where

E is the expectation over the distribution of the trajectory $(s_0, a_0, s_1, a_1, \cdots)$. Q^π can be determined by solving the Bellman equation [3]:

$$Q^\pi(s,a) = E^\pi\left[r(s,a) + \gamma Q^\pi(s',a')\right] \tag{1}$$

Experience replay Buffer $\mathcal{B}$ is a collection of agent store experiences, which is used to update the neural network. In our work, each experience $\langle s, a, r, s' \rangle$ is stored in the corresponding buffer according to the reward classification when storing, and then we obtained some sub-buffers $(\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3, \cdots)$. We propose a new sampling method to get the ratio of samples from each sub-buffer $\mathcal{B}_i, i = 1, 2, 3, \cdots$, and then obtain the corresponding number of samples.

4 Methods

In the sparse reward environment, agent obtains few useful rewards, so it is very important to make the best use of these rewards. Using an experience replay buffer leads to two design problems: how to store experience, and how to replay experiences. In this paper, we propose a Novel State Space Exploration (NSSE) method, where multi-buffer stores experience separately and priority is used to represent the learning situation of each buffer to determine the sampling ratio from each buffer. The following are the details of how we solve the above two problems.

4.1 Multiple Replay Buffer

Generally, the experiences obtained by the agent over a period of time are stored in the replay buffer. We use multiple buffers to store experiences, and priorities to clear the sampling ratio from each buffer. This has a great advantage: the agent can learn better by increasing the replay probability of the sparse rewards.

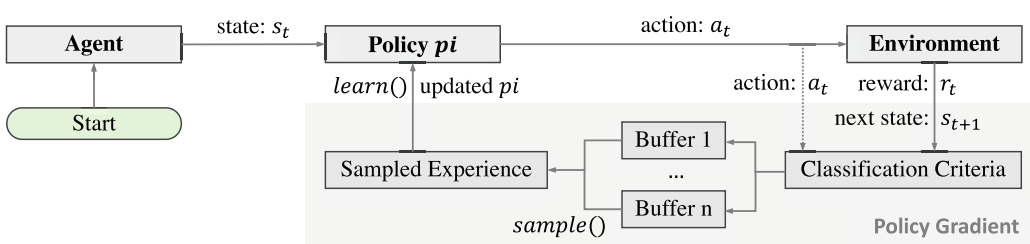


Fig. 1. The overview of the method of grouping reply buffers.

The schematic view of our algorithm is shown in Fig. 1. There are four important components including (1) Policy π which takes as input the state s_t, then produces action a_t to interact with the (2) Environment. The environment returns a new state s_{t+1} and the reward r_t to the agent. The state s_t,

action a_t, reward r_t, and new state s_{t+1} are respectively stored into the corresponding sub-buffer according to (3) Classification Criteria. When learning, our new method is adapted to sample from all sub-buffers, and then we obtained (4) sampled experience to update our neural network.

4.2 Asynchronous Priority Method

Inspired by the prioritised sweeping method, the experiences sampled from each sub-buffer are determined by a sampling ratio based on the temporal difference (TD) error. The TD-error reflects the difference between the predicted reward and the actual reward, serving as an indicator of the agent's learning progress for each sub-buffer. If the TD-error calculated from the sampling data in a particular sub-buffer is larger, it indicates that the experience within that sub-buffer is more valuable for the agent's learning. Therefore, it is necessary to use more experience in this sub-buffer during the training of the neural network. Due to the large amount of calculation of the experience priorities in each sub-buffer, we utilise the sample mean to estimate the population mean, and this is statistically rational. The specific sampling method is as follows.

For every sub-buffer $\mathcal{B}_i, i = 1, 2, \cdots, n$, where n is the number of sub-buffers, we sample a batch of data $\{(s_{ij}, a_{ij}, r_{ij}, s_{ij}{}')\}, j = 1, 2, \cdots, m$ from $\mathcal{B}_i$, where m is the size of the batch. For every data, according to the Eq. 1, we use the target network Q^{π^*} to calculate

$$y_{ij} = r_{ij} + \gamma max_{a_i} Q^{\pi^*}(s_{ij}{}', a_i) \tag{2}$$

We define the sampling ratio $ratio_i$ of the batch $\mathcal{B}_i$ as $ratio_i = \delta_i / \sum_{k=1}^{n} \delta_k$, where δ_k is the TD-error calculated by the average of $|Q^{\pi}(s_{kj}, a_{kj}) - y_{kj}|, j = 1, 2, \cdots, m$. Thus, we obtain $num_i = \lfloor ratio_i \times m + 1/2 \rfloor$ as the number of samples in sub-buffer $\mathcal{B}_i$. Due to the accuracy problem, the above calculation method can be used for the number of the first $n-1$ sub-buffers, and the number of the last buffer, we can obtain from $m - \sum_{k=1}^{n-1} num_k$. Then we obtain a minibatch by sampling the corresponding number of samples from every sub-buffer by uniform sampling. It should be noted that it is unnecessary to update $ratio_i$ frequently, we can update it once in a while. After that, we use the newly sampled data to update the policy π.

5 Experiment

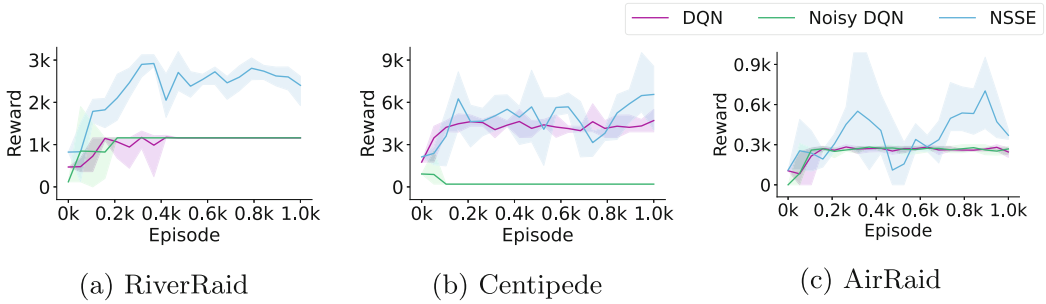
We selected three typical discrete sparse reward environments in Atari 2600 [2], RiverRaid-v5, Centipede-v5, and AirRaid-v5. In our experiment, every experiment contains three algorithms (three curves), and We test the performance of the agent every 10 episodes. We considered two baseline algorithms that use uniform experience replay buffers. One is the DQN algorithm [7], and the other is the Noisy DQN algorithm [6]. The neural network contains three Convolutional

layers and two Linear layers. For Noisy DQN, we add factorised Gaussian noise in the Linear layers.

In this paper, the most relevant component of these baseline algorithms is the experience replay buffer. The replay buffer is uniformly set to a queue with a size of 1000. The baseline algorithms process a minibatch of size 16 sampled uniformly from the replay buffer. We can divide the experience into 2 sub-buffers. The first sub-buffer store experience with rewards greater than 0, and the second one is equal to 0. The initial sampling ratio is both $\frac{1}{2}$.

We compared our novel state space exploration with two baselines (DQN and Noisy DQN) on the three Atari Environments (RiverRaid, Centipede, and AirRaid). The applied evaluation metric is the total reward for interacting with the environment in each episode. As shown in Fig. 2, our algorithm performed better than the two baselines. While in Fig. 2(a) RiverRaid and Fig. 2(b) Centipede, part of the reward curves become a line and no longer change. The main reason is that the policy gradient update is too slow, causing the gradient to disappear. Some oscillations are observed in our experiment, especially in Fig. 2(c) AirRaid, the likely cause is q value overestimation, which results in significant errors in neural network parameters. These errors, in turn, have an adverse effect on policy learning.

Additionally, we also provide a system performance analysis. Compared with the method of applying Priority Experience Replay (PER) buffer [13], since no storage prioritisation is required, our method NSSE saves storage requirements by around 50%.



(a) RiverRaid (b) Centipede (c) AirRaid

Fig. 2. Experimental Test Results in Atari.

6 Conclusion

This paper presents NSSE, a new state space exploration method for the discrete sparse-reward environment. By dividing the discrete rewards into sub-buffers based on their sparsity and calculating the sampling ratios of these sub-buffers using TD-error, the proposed method enables better learning efficiency and system performance. We evaluate our method on three classical Atari environments.

The results show that our method has superior learning efficiency compared to DQN and NoisyDQN, with lower memory consumption compared to PER. In the future, we will extend our work to more practical environments.

References

1. Aubret, A., Matignon, L., Hassas, S.: An information-theoretic perspective on intrinsic motivation in reinforcement learning: a survey. Entropy **25**(2), 327 (2023)
2. Bellemare, M.G., Naddaf, Y., Veness, J., Bowling, M.: The arcade learning environment: an evaluation platform for general agents. J. Artif. Intell. Res. **47**, 253–279 (2013)
3. Bellman, R.: A Markovian decision process. J. Math. Mech. 679–684 (1957)
4. Burda, Y., Edwards, H., Storkey, A., Klimov, O.: Exploration by random network distillation. arXiv preprint arXiv:1810.12894 (2018)
5. Ecoffet, A., Huizinga, J., Lehman, J., Stanley, K.O., Clune, J.: First return, then explore. Nature **590**(7847), 580–586 (2021)
6. Fortunato, M., et al.: Noisy networks for exploration. arXiv preprint arXiv:1706.10295 (2017)
7. Hester, T., et al.: Deep Q-learning from demonstrations. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32 (2018)
8. Horgan, D., et al.: Distributed prioritized experience replay. arXiv preprint arXiv:1803.00933 (2018)
9. Jo, D., et al.: LECO: learnable episodic count for task-specific intrinsic reward. Adv. Neural. Inf. Process. Syst. **35**, 30432–30445 (2022)
10. Pathak, D., Agrawal, P., Efros, A.A., Darrell, T.: Curiosity-driven exploration by self-supervised prediction. In: International Conference on Machine Learning, pp. 2778–2787. PMLR (2017)
11. Puterman, M.L.: Markov decision processes: discrete stochastic dynamic programming. In: Wiley Series in Probability and Statistics (1994)
12. Saglam, B., Mutlu, F.B., Cicek, D.C., Kozat, S.S.: Actor prioritized experience replay. arXiv preprint arXiv:2209.00532 (2022)
13. Schaul, T., Quan, J., Antonoglou, I., Silver, D.: Prioritized experience replay. arXiv preprint arXiv:1511.05952 (2015)
14. Seo, Y., Chen, L., Shin, J., Lee, H., Abbeel, P., Lee, K.: State entropy maximization with random encoders for efficient exploration. In: International Conference on Machine Learning, pp. 9443–9454. PMLR (2021)
15. Yu, X., Lyu, Y., Tsang, I.: Intrinsic reward driven imitation learning via generative model. In: International Conference on Machine Learning, pp. 10925–10935. PMLR (2020)
16. Yuan, M., Pun, M.O., Wang, D.: Rényi state entropy maximization for exploration acceleration in reinforcement learning. IEEE Trans. Artif. Intell. (2022)
17. Zheng, C., Yang, S., Parra-Ullauri, J.M., Garcia-Dominguez, A., Bencomo, N.: Reward-reinforced generative adversarial networks for multi-agent systems. IEEE Trans. Emerg. Top. Comput. Intell. **6**, 479–488 (2021)